



Caching for Fast Inference

Reducing latency and cost by reusing past results

A practical guide for engineers implementing low-latency ML systems

Why Caching Matters

Inference is **expensive** in terms of GPU time, memory, and power consumption.

Many inference requests are **repeated or highly similar**, especially in production environments.

Strategic caching delivers three critical benefits:

1 Latency

Near-instant responses improve user experience

2 Cost

Fewer GPU cycles means lower infrastructure bills

3 Load

Reduced strain provides more headroom and system stability



Levels of Caching

Effective caching strategies operate at multiple levels in your inference stack:



Client-side

Browser/app caches store results locally, eliminating network requests entirely



Edge/CDN

Store responses geographically close to users, reducing latency



API Gateway

Cache REST/gRPC calls before they reach your application servers



Application Layer

Redis/Memcached store inference outputs for fast retrieval



Feature/Model Cache

Store embeddings and intermediate computational results

What to Cache?



Not all inference data is equally valuable for caching. Focus on these high-ROI elements:

- **Full API responses** for idempotent queries (same input always produces same output)
- **Model outputs for identical inputs**, especially for expensive operations
- **Precomputed embeddings** or vector search indices that change infrequently
- **Preprocessing results** like tokenized text or resized images

❏ Consider caching at multiple stages of your inference pipeline for compound benefits.

Cache Keys & Strategies

Deterministic Keys

Generate consistent hash values from input payloads to ensure cache hits

```
cache_key = hash(normalize(request.payload) +  
model_version)
```

Input Normalization

Remove irrelevant variations that don't affect results:

- Strip extra whitespace
- Sort JSON fields
- Standardize text case

Version Tagging

Always include model version in cache keys to prevent serving stale results after model updates

Time-to-Live (TTL)

Balance freshness vs. cost savings by setting appropriate expiration periods based on data volatility

Effective key design is critical—too specific and you miss cache opportunities; too general and you serve incorrect results.

Tools & Implementations



In-Memory Stores

Redis / Memcached provide microsecond-level access times for frequently used inference results

Edge Solutions

Cloudflare, Akamai, AWS CloudFront deliver cached responses from locations near users

Gateway Layers

Envoy, NGINX with response caching intercept and serve repeated requests before they hit your application

Vector Databases

FAISS, Pinecone, Weaviate cache nearest neighbor results for embedding-based systems

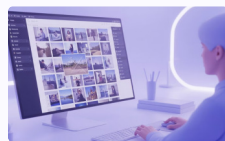
Inference-Specific Patterns

Different ML models benefit from specialized caching approaches:



LLMs

Cache prompt → response pairs, especially for retrieval + summarization tasks that produce deterministic outputs



Computer Vision

Cache image fingerprints → classification results to quickly identify previously analyzed visual content



Recommendation

Cache top-N recommendation lists per user/segment to avoid recalculating frequently accessed suggestions



Search

Cache embedding → ranking results to speed up semantic search operations with similar queries

Challenges & Pitfalls

Staleness

Cached results may not match updated models or data, creating inconsistent user experiences

Memory Pressure

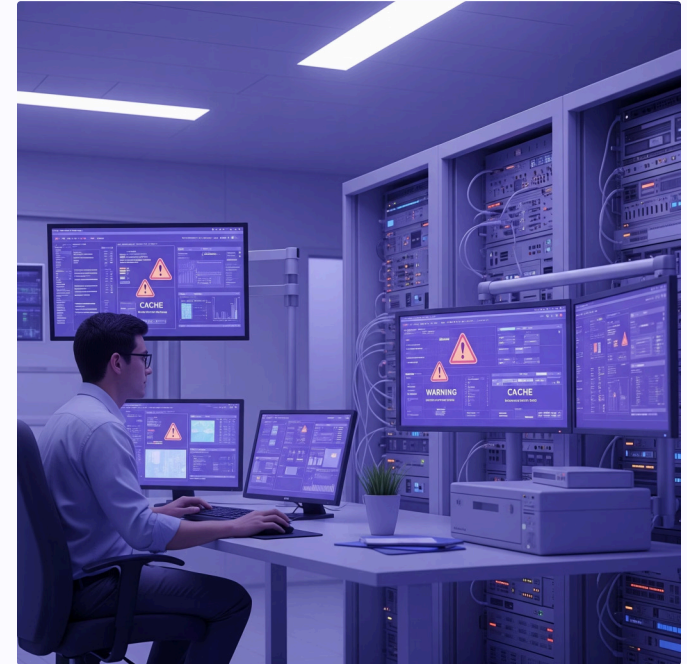
Large outputs like embeddings or images can quickly consume available cache space

Cache Invalidation

"There are only two hard things in Computer Science: cache invalidation and naming things."

Metric Masking

Overcaching can hide real drift in production metrics, delaying detection of model degradation



⊗ Inadequate cache invalidation is the #1 source of hard-to-debug inference issues in production.

Best Practices

01

Version Everything

Always include model version in cache keys to ensure cache invalidation when models change

02

Implement Tiered Caching

Layer caches (edge → Redis → local) to maximize hit rates and minimize latency

03

Monitor Aggressively

Track hit/miss ratios and eviction patterns to optimize cache policies

04

Balance TTL Settings

Match expiration times to data volatility—shorter for rapidly changing data, longer for stable outputs

05

Quantify Benefits

Log cache usage patterns to analyze ROI and justify infrastructure investments

Key Takeaways

Caching is a first-class infrastructure strategy for inference systems, not an afterthought.

- Strategically implemented caching dramatically reduces **latency, cost, and stress** on GPU clusters
- Combine **multiple cache layers** for maximum efficiency and resilience
- Design cache invalidation and monitoring systems **upfront**, not as add-ons
- Start with high-value, deterministic inference paths for quick wins

Questions? Contact the ML Infrastructure team at ml-infra@company.com



- ✔ Teams implementing these strategies have seen up to 70% reduction in inference costs and 5x latency improvements.